

Customers (*continued*)

- playing role of “bad customer” in exploratory testing, 166–167
- teams and, 416
- tools for engaging. *See* Tools, for customer engagement
- valuing customer collaboration over contract negotiation, 151–152

Cycle time

- defined, 416
- DevOps shortening, 361–362
- reducing, 367
- setting goal for, 387–388

D

Dashboard, visibility to tests and test results, 392

Data

- in 7 Product Dimensions, 130
- business intelligence and, 352–353
- challenges of Big Data, 357, 359
- managing test data, 249–250, 355–356
- solving bad test data problem, 353–354

Data integrity tests, skill requirements for DW/BI testers, 348

Data modeling

- abstraction of logical models, 351
- for data warehouse, 347

Data warehouses (DWs)

- applying agile principles to, 351
- challenges of Big Data, 357, 359
- data in, 352–353
- learning to test, 351–352
- managing test data, 355–356
- solving bad test data problem, 353–354
- succeeding with agile testing in, 400
- testing performance and scale, 357–359
- unique aspects of testing in, 347–350

Database team, managing internal dependencies, 292

Databases

- DW/BI tester skills, 348
- general technical skills, 59

de Bono, Edward, 50

Deadlines

- building trust and, 17
- pressure of unrealistic, 7–8

Debrief sessions

- following testing tours (Gärtner), 175
- recording results of exploratory testing, 186
- SBTM use for training testers, 176

Debugging. *See* Defects/bugs

Decision tables, test design techniques, 67

Default Data pattern (Morgan), 250

Defects/bugs

- “bug hunts,” 183
- catching/tracking, 392
- debugging test failures, 250–251
- implementing agile at Dell and, 281
- making test maintenance visible, 213
- preventing, 36, 47, 102, 158, 216, 362
- prioritizing fixes, 289
- processes for dealing with, 287
- reducing defect debt (zero tolerance), 213–216
- visual (whiteboard) approach to, 81–82

Defect-tracking systems (DTSs), 321

Definitions/assumptions, in collaboration process, 52

Deliverables

- formal documentation of, 92–94
- impact mapping and, 124–125

Delivery. *See also* Product release

- delivery (development) team, 416–417
- product delivery and, 296–297
- single and multiple-team cycles, 91

Dell example, of agile journey

- challenges and solutions, 279–280
- evolution of, 282
- implementing, 281
- results, 296
- scaling agile testing, 287–288

Dependencies

- coordinating between teams, 285
- customer involvement in enterprises, 294–296
- distributed teams, 305
- managing, 292
- removing, 293
- third-party partnerships, 292–294

Derby, Esther

- adding feedback loops and, 16
- Problem Solving Leadership (PSL) course, 74

Design patterns and principles

- Default Data pattern (Morgan), 250

- Factory pattern, 410–412
 - overview of, 240
- Page Object pattern, 246–248, 407–410
 - testing through the API, 241–243
 - testing through the UI, 243–246
- Desktop sharing, collaboration tools, 321
- Detail, getting bogged down in, 159–160
- Detail level in planning. *See* Levels of precision (detail), for planning
- Development. *See also* DevOps
 - core practices, 394
 - environment for, 59–60
 - guiding with examples, 55–56, 148–149
 - liability in separating from IT or operations, 365
 - need for testers in, 26–27
 - planning iterative development, 88
 - tests guiding, 102
- Development (delivery) teams
 - adopting agile values, 7
 - bonding with, 345
 - collaborating to meet regulatory needs, 344–345
 - defined, 416–417
- Development, building the right thing
 - 7 Product Dimensions (Gottesdiener and Gorman), 129–131
 - determining the purpose of new features, 121–123
 - example applying 7 Product Dimensions, 131–133
 - impact mapping, 123–126
 - investing and, 134–135
 - overview of, 119–120
 - story mapping, 126–129
 - tools for customer engagement, 123
 - tools for exploring early, 134
- DevOps
 - adding infrastructure to testing scope, 365–367
 - in all Quadrants, 363–364
 - automating build verification testing, 369–371
 - automating provisioning of configuration base states, 374–376
 - branching issues and, 63
 - build pipeline and, 367–368
 - defined, 417
 - evolution of agile testing and, 5
 - interdependence of software development and infrastructure and operations, 26
 - monitoring and logging and, 59
 - overview of, 361–362
 - quality and, 363
 - testers adding DevOps value, 371–372
 - testing infrastructure, 372–373
- DevOps for Developers* (Hüttermann), 361
- Dinwiddie, George, xxix, 4
- Discover to Deliver* (Gottesdiener and Gorman), 107, 191–192
- “Discuss-Distill-Develop-Demo” cycle, in ATDD, 276
- Dispersed teams. *See also* Distributed teams
 - communication and collaboration and, 310
 - overview of, 299–301
 - visualizing testing in, 386
- Distributed teams
 - challenges of, 302
 - collaboration and, 309–312, 400
 - collaboration tools, 319–321
 - communication and, 309–311, 400
 - communication tools, 319
 - coping strategies, 308
 - cultural issues, 302–303
 - dependencies, 305
 - experience of working on, 306–307
 - experience working on offshore test team, 315–317
 - facilitating online communication, 306
 - integrating, 308–309
 - language issues, 303–304
 - management issues, 317–318
 - offshore testing, 312–317
 - overview of, 299–302
 - planning sessions, 305, 308
 - reasons for, 301–302
 - short feedback loops, 321–322
 - time zone issues, 304–305
 - tools for, 319–322
 - visualizing testing, 386
- Documentation. *See also* Living documentation
 - of deliverables, 92–94
 - excessive, 341–342
 - “lack of documentation” myth, 339–340
 - linking to requirements, 345

Documentation (*continued*)
 recording results of exploratory testing, 185
 skill requirements, 141–142
 training end users, 295

Domain knowledge, 46–47

Domain-specific language (DSL)
 collaborating with distributed teams, 311
 describing feature behavior, 147
 example use and, 157–158
 guiding development with customer-facing tests, 239
 programmer buy-in and, 160
 tests using, 56
 in whole team approach, 259

Done
 Feature Done, 286
 Product Done, 287
 Release Done, 342
 Story Done, 286

Don't repeat yourself (DRY) principle, 35, 240

Driving development. *See* Examples, guiding development

Dropbox
 collaboration tools, 312
 experience working on offshore test team, 315–317

DRY (Don't repeat yourself) principle, 35, 240

DSL. *See* Domain-specific language (DSL)

DTs (Defect-tracking systems), 321

DWs. *See* Data warehouses (DWs)

E

Elaboration, types of thinking skills, 49

Eliot, Seth, 200–201

Email, as communication tool, 319

Embedded systems/software
 automation-assisted exploratory testing (Hagar), 334–335
 critical nature of testing in, 328–329
 defined, 325
 learning agile testing for mobile software (Harrison), 336–337
 lessons learned in applying agile testing (Hagar), 330–332
 overview of, 325–326
 similarities and differences of agile testing in, 326–328
 test automation strategies (Morgan), 332–334
 testing context and, xxvi
 types of agile approaches used with, 329

Emery, Dale, 248

Emotions, learning and, 70–71

Empathy, giving/receiving feedback and, 45

End game
 activities, 285
 additional testing during, 90
 addressing integration issues early, 291
 defined, 417
 taking release to Product Done, 287

End-to-end testing
 Ruhland on, 193
 susceptibility to change, 229–230
 test automation pyramid, 225

Enterprise Solutions Group (ESG), at Dell
 challenge of hardware compatibility matrix, 369, 371
 challenges in transitioning to agile, 266–267, 279–280
 results of adopting agile, 296

Enterprises (large organizations)
 aligning target setting for test automation across, 277–278
 Big Data as challenge to, 357
 coordinating multiple teams, 283–284
 coordinating tooling for, 289–290
 customer involvement, 294–296
 defined, 417
 dependency management, 292
 implementing test automation in, 254–258
 organizational controls, 278–283
 overview of, 275
 product delivery and, 296–297
 scaling automation for, 264–265, 267–268
 scaling testing for, 276–277
 system test team and environment and, 284–289
 test coverage, 291
 third-party partnerships, 292–294
 version control, 290
 what they are, 275–276

Environments
 in 7 Product Dimensions, 130

- in DevOps, 376
- development environments, 59–60
- feature testing by, 286
- system test team and, 284–289
- test environments. *See* Test environments
- ESG. *See* Enterprise Solutions Group (ESG), at Dell
- ETL. *See* Extract, transform, and load (ETL)
- Evangelisti, Augusto, xxix
 - about the contributors section, xxxvi
 - on branching strategy, 63
 - on conversation value, 150
 - on managing testers, 19
 - on quality guilds, 14
 - on reducing defect debt, 213–216
- Evans, David
 - about the contributors section, xxxvi
 - on automated tests as living documentation, 339
 - on getting to know distributed team members, 310
 - “Pillars of Testing” model, 402–405
 - on spending time effectively, 9
- Everyday Scripting with Ruby* (Marick), 57
- Examples, guiding development
 - ATDD, 149–150
 - BDD, 152–153
 - benefits of, 157–158
 - capturing examples and turning into automated tests, 147
 - “checked examples” in Q1 and Q2 tests, 103
 - collaborating with distributed teams, 311
 - conversation as means of delivering value, 150–152
 - as core practice, 394–395
 - in defect prevention, 158
 - evolution of agile testing and, 3
 - overview of, 55–56, 145
 - pitfalls of, 159–162
 - planning Q2 tests and, 105
 - power of using, 145, 147–148
 - resource materials for learning mechanics of, 162
 - SBE, 153–154
 - solving bad test data problem, 354
 - use in real life, 146–147
 - various approaches to, 148–149
 - where to get, 155–157
- Executives, educating, 17–19
- Experiments
 - learning and, xxviii
 - small experiments in continual learning, 80
- Exploratory testing
 - in agile testing context, 188–190
 - automation-assisted, 334–335
 - collaboration with development team and, 345
 - as core practice, 395–396
 - creating test charters, 94, 168–171
 - defined, 417
 - in DW/BI, 350
 - in evolution of agile testing, 5
 - generating test charter ideas, 171
 - groups in, 183–185
 - helping others and, 80
 - Hendrickson on, 66–67
 - journeys in, 175–176
 - managing test charters, 176
 - offshore test teams and, 315
 - overview of, 165–167
 - personas in, 171–174
 - Q3 tests (business-facing tests) and, 66, 103, 106–107
 - recording results, 185–188
 - in SBTM, 176–178
 - skill development for, 167
 - skills requirements in square-shaped teams, 30–32
 - stories in, 175
 - in TBMT, 178–183
 - in test automation pyramids 223, 227–228
 - tours in, 174–175
- Explore It* (Hendrickson), 50, 66, 106, 168
- Extract, transform, and load (ETL)
 - defined, 417
 - learning to test BI and, 351
 - managing test data and, 355–356
 - speeding up/automating, 349
 - testing data rules, 353–354
- Extreme Manufacturing, 331
- Extreme Programming (XP)
 - agile testing quadrants and, 101
 - evolution of agile testing and, 3–4
 - testing needs and, 192

- F**
- Facilitation skills, thinking skills, 42–43
 - Factory pattern, design patterns, 410–412
 - Fail fast, learning and, 12
 - Farley, David, 361
 - Fast feedback, agile principles
 - API level tests and, 223
 - applying to DW/BI, 351–352
 - automated tests and, 97, 153, 161, 227
 - automation levels and, 115
 - A/B tests and, 204
 - build pipeline and, 416
 - DevOps and, 362
 - in distributed teams, 301
 - learning from, 12, 87, 394
 - as objective of test execution, 256
 - prioritizing information from, 347
 - for product improvement and visibility, 17
 - providing information to developers, 331
 - rapid release cycles and, 200
 - on regression failures, 234, 289
 - for risk reduction, 291
 - test environments and, 60
 - in testing and coding approach, 354
 - “Trinity Testing” (Harty) and, 184
 - from users, 327
 - Fazal, Kareem
 - about the contributors section, xxxvi
 - on automating build verification testing, 369–371
 - Fearless Change* (Manns and Rising), 19
 - Feature Done, 286
 - Feature injection, BDD and, 153
 - Feature testing
 - as core practice, 396–397
 - in different environments, 286
 - iterative testing and fast feedback and, 5
 - release planning and, 92
 - remember the big picture, 396–397
 - story maps in, 126
 - Features
 - exploratory testing at feature level, 189
 - levels of precision in planning, 92–94, 96
 - overview of, 88
 - planning Q2 tests and, 105
 - planning Q3 tests and, 106–107
 - prioritizing, 283
 - the “why” of new features, 121–123
 - Federated data, 348, 417
 - “Feedback Wrap” workout (Appelo), 45
 - Feedback/feedback loops
 - continuous integration and, 262
 - fail fast and, 12
 - fast feedback. *See* Fast feedback, agile principles
 - impact mapping and, 125
 - key success factors, 393
 - mobile and embedded systems and, 326
 - offshore test teams and, 316–317
 - organizational culture and, 15–17
 - planning for, 87
 - from prototyping, 205–206
 - regression testing and, 161
 - short feedback loops in agile testing, 321–322
 - skill in giving/receiving, 45–46
 - test automation pyramid and, 223
 - Fishbone diagrams. *See* Ishikawa diagrams, 44
 - FIT (Framework for Integrated Test), 3
 - FitNesse tool, 254–255, 257, 265
 - Flow diagrams
 - benefits of flow-based systems, 383
 - example use in, 145
 - kanban and, 88
 - story level and, 96
 - Focus groups, 140
 - Framework for Integrated Test (FIT), 3
 - Freeman, Steve, 126
 - Frempong, Benjamin
 - about the contributors section, xxxvii
 - on automating provisioning of configuration base states, 374–376
 - Functional decomposition, types of thinking
 - skills, 49
 - Functional tests
 - ATDD and, 150
 - in expanded test automation pyramid, 233–234
 - in test automation pyramid, 115
 - tools for, 237
 - turning examples into automated tests, 147
 - Functionality
 - key components of business value of software, 122
 - testing through the UI, 243
 - FURPS model, for planning tests, 113–114

G

- g11n. *See* Globalization
- Gärtner, Markus, xxix
 - on ATDD, 56
 - on debriefing following tours, 175
 - Pomodoro Testing, 182–183
 - on starting testing process right, 239
 - on technical debt, 211
- Gawande, Atul, 9
- Generalizing specialists, 33–36
- George, Chris
 - about the contributors section, xxxvii
 - on reducing technical debt, 218–220
- Gilb, Kai, 107
- Gilb, Tom, 107, 277
- Git tool, 365–366
- Globalization
 - challenges of global markets, 195
 - defined, 415
 - guiding development with examples, 55–56, 148–150
 - language and character set support, 198
 - planning Q4 tests and, 107
 - programmer buy-in and, 160
- Goals
 - in collaboration process, 52
 - metrics in setting, 277
 - SMART, 49, 386
 - stakeholders in goal setting process, 124
- Gorman, Mary, xxx
 - 7 Product Dimensions, 129, 131–133
 - about the contributors section, xxxvii
 - given_when_then template for getting examples, 156
 - on Planguage, 107
 - on representation of quality attributes, 191
- GoToMeeting, collaboration tool, 312
- Gottesdiener, Ellen, xxx
 - 7 Product Dimensions, 129, 131–133
 - about the contributors section, xxxvii
 - given_when_then template for getting examples, 156
 - on Planguage, 107
 - on representation of quality attributes, 191
- Graphs, for recording test results, 187
- Group chats, collaboration tool, 321

Group hugs

- concurrency testing and, 194–195
 - exploratory testing and, 183–184
- Groups, in exploratory testing, 183–185
- Guest, David, 29
- Guiding development. *See* Examples, guiding development

H

- Hagar, Jon
 - about the contributors section, xxxviii
 - on automation-assisted exploratory testing, 334–335
 - defining embedded software and mobile apps, 325
 - on lessons learned applying agile testing to mobile and embedded systems, 330–332
- Hagberg, Jan Petter, 303, 322
- Hardware compatibility matrix, at Dell, 369
- Hariprasad, Parimala
 - about the contributors section, xxxviii
 - on offshore testing teams, 315–318
- Harrison, JeanAnn, xxix
 - about the contributors section, xxxviii
 - on collaboration in meeting regulatory needs, 344–346
 - on learning agile testing for mobile software, 336–337
- Harty, Julian, 330
 - on critical nature of testing for mobile apps, 328
 - on “Trinity Testing,” 184
- Hassa, Christian, 276–277
- Heinrich, Mike
 - about the contributors section, xxxviii
 - example of data warehouse setup—
Figure 22-1, 348
 - on learning to test BI, 351–352
- Heinze, Sherry, xxix
 - about the contributors section, xxxvix
 - on example use in everyday life, 146–147
- Hellesøy, Aslak, 56
- Help Desk persona, 172–173
- Hendrickson, Elisabeth, xvii, 109, 214
 - “Agile Acid Test,” 79
 - ATDD and, 151
 - challenging use of Quadrants, 112–113

- Hendrickson, Elisabeth (*continued*)
 “Discuss-Distill-Develop-Demo” cycle, 276
 evolution of agile testing and, 3
 on exploratory testing, 66–67
 on planning Q3 tests, 106–107
 on resources for good charters, 168
 “Test Heuristics Cheat Sheet” as source for ideas
 in exploratory testing, 167
 on tools for structured and focused thinking, 50
- Heuristics
 defined, 418
 in exploratory testing, 166
 planning models and, 114
- Heusser, Matthew
 about the contributors section, xxxvix
 on charters and session-based testing, 169–170
 on managing regression tests, 177–178
- Hiring
 finding the right people, 36–37
 onboarding process for testers, 37–38
- Hiring Geeks That Fit* (Rothman), 37
- How Google Tests Software* (Whittaker), 109
- “How?” questions
 in creation of roadmaps, 123–124
 iPhone example, 140
- Humble, Jez, 361
- “Hump of pain,” in test automation, 237
- Hunt, Andy, 49–50
- Hussman, David, 71
 on journeys as means of creating charters,
 175–176
 persona use by, 171
- Hüttermann, Michael, 372
 about the contributors section, xxxvix
 on adding infrastructure to testing scope,
 365–367
 on DevOps, 361
 on “outside-in, barrier-free collaborative” in Q
 tests, 103–104
- Hypotheses, A/B tests for checking, 134
- I**
- i18n. *See* Internationalization
- IaaS (Infrastructure as a service), 418
- IDEs. *See* Integrated development environments
 (IDEs)
- Impact mapping
 overview of, 123–126
 for product vision, 89
 tools for visualization of thinking process,
 43–44
- Infrastructure
 adding infrastructure to testing scope, 365–367
 “Levels of Service” provided by infrastructure
 testers, 371–372
 support for distributed teams, 322
 testing, 372–373
 testing requirements and, 192
- Infrastructure as a service (IaaS), 418
- Initiative, testers taking, 142–143
- Integrated development environments (IDEs)
 collaboration and, 56
 manage automation with, 262
 tester expertise with IDE tools, 60
 T-shaped skills and, 29
 unit tests in, 4
- Integration
 addressing integration issues early, 291
 of distributed teams, 308–309
 product release level and, 90
 system integration team in offshore testing, 313
- Integration tests, offshore testing and, 312–314
- Integrity checks, data and relational, 59
- Interfaces
 in 7 Product Dimensions, 129–130
 technical skills and, 58
 user interfaces. *See* User interface (UI)
- Internationalization
 planning Q4 tests and, 107
 testing needs and, 195–200
- Interoperability, test planning and, 91
- Investigative testing
 A/B tests. *See* A/B tests
 concurrency testing, 194–195
 exploratory testing. *See* Exploratory testing
 internationalization and localization, 195–200
 overview of, 163
 regression testing. *See* Regression tests
 types of, 164
 user acceptance testing. *See* User acceptance
 testing (UAT)
 user experience testing. *See* User experience (UX)

Ishikawa diagrams
 types of thinking skills, 49
 for visualization of thinking process, 44
IT departments, liability in separating from
 development, 365

Iterations
 iteration demos as learning opportunity, 77
 managing Big Data, 358
 planning, 88, 93, 96–97, 130, 293, 310
 planning meetings, 173–174, 186
 regression testing and, 98
 stories and, 28
 visual cues of technical debt, 217
Ivarsson, Anders, 297

J

JavaScript, test automation pyramid and, 226
JBehave, for automating acceptance tests, 214
Jdefault library, 250
Jenkins tools
 in DevOps example, 366
 testing infrastructure with, 373
Job responsibilities. *See* Roles/competencies;
 Testers, determining job responsibilities
Johnson, Karen N., 330
Jones, Griffin
 about the contributors section, xxxvix
 on possibility of agile in regulated
 environment, 343
Journeys, in creating test charters, 175–176
JUnit tests, 265

K

Kahneman, Daniel, 50
Kämper, Stephan
 about the contributors section, xl
 on testing infrastructure, 372–373
Kanban
 for capacity utilization (Zheglov), 10–12
 as communication tool, 319
 for continuous improvement, 386–389
 course on, 75
 defined, 418
 distributed teams and, 311
 as flow-based method, 88
 in reducing defect debt, 214

 use in testing (Rogalsky), 382–385
 for visualizing testing, 382–385
Kaner, Cem, 399
Karten, Naomi, 49
Keeping it real
 continual improvement and adaptation, 6
 as core practice, 401
 cutting corners and, 220
 getting buy-in, 160
 maintaining automation tests, 248
 making regulatory requirements part of
 work, 344
 managing source code, 64
 options in dealing with uncertainty, 18
 planning and, 87
 regression testing and, 200
 tests, skills, and resources and, 104–105
 trust building within teams, 45
 unsustainable pace and, 8
 value of whole team understanding operating
 environment, 376
 “what” vs. “why” in feature development, 122
 working with third-parties and, 293
Kelln, Nancy, 25
Keogh, Liz, xxix, 18
 on BDD, 152
 on getting needed capabilities in place before
 testing, 239
 on having too many unknowns, 161–162
 on reviewing workflow for simplicity, 385
 on tests for monitoring system performance,
 107–108
 on value of conversations, 147
Kerievsky, Joshua, 3
Khoo, Trish
 about the contributors section, xl
 on need for testers in development, 26–27
Kniberg, Henrik
 on use of agile by Spotify, 16
 using agile principles without being too
 rigorous, 297
Knight, Adam, xxix
 about the contributors section, xl
 on branching strategy, 64–65
 on development environments, 59
 DW/BI tests and, 350

Knight, Adam (*continued*)
 experience using TBTM, 182
 on managing testers, 19
 on square-shaped team, 30–32
 on testing performance and scale, 357–359
 on tools for extending test harnesses, 237–238

Kohl, Jonathan, 326, 330

L

L10n. *See* Localization

Lambert, Rob, 29

Langr, Jeff

Arrange-Act-Assert pattern, 240
 automating acceptance tests, 214
 mnemonic for nonfunctional requirements—
 FURPS, 113

Large organizations. *See* Enterprises (large organizations)

Leadership skills, 74

Lean Coffee sessions, 14

Lean principles

implementing, 276
 just in time and, 85
 in manufacturing, 10
 startup, 108–110, 134, 203

Learning. *See also* Continual learning

change and, 9
 conferences, courses, meet-ups, coach camps
 and collaborating, 72–75
 domain knowledge, 46–47
 evolution of agile testing and, 5
 experiments and, xxviii
 fostering a learning culture, 13–15, 77
 helping others, 79–80
 importance of a learning culture, 12–13
 making time for, 8–12, 77–79
 mentors, learning from, 71
 overview of, 69
 publications, podcasts, and online communities,
 75–77
 resources, 72–74
 roles and competencies and. *See* Roles/
 competencies
 social networking, 74
 styles, 69–72
 surprise learning, 80–83

technical skills. *See* Technical awareness
 (technical skills)

to test business intelligence, 351–352

thinking skills. *See* Thinking skills

Legacy systems

defined, 418
 internationalization requirements, 198
 reducing technical debt, 218
 session-based testing and, 169
 test strategy for, 177

Levels of precision (detail), for planning

applying to enterprise example, 284
 different points of view and, 87–89
 feature level, 92–94, 96
 overview of, 87
 product delivery cycle, 91–92
 product release level, 89–90
 regression testing and, 97–98
 story level, 96
 task level, 96–97
 visualizing what you are testing, 98–100

Listening

learning by, 69
 thinking skills, 48–49

Living documentation. *See also* Documentation

continuity provided by domain-specific
 language, 239
 SBE and, 153–154, 420
 TDD and, 56
 test automation and, 38, 209, 268, 339, 342
 testing through the API and, 241–243
 testing through the UI and, 243

L-mode (linear and slow), thinking skills
 and, 49

Load testing

applying Q4 tests, 103
 test environments, 61

Localization

planning Q4 tests and, 107
 testing needs and, 195–200

Log files, reading, 59

Logic Bubbles, thinking skills and, 50

Lyndsay, James, xxix, 178

on breadth and specificity of test charters, 169
 resources for session-based testing, 178
 on scripted testing vs. exploratory testing, 166

M

- Maassen, Olav, 123
- Maintainability, in expanded test automation pyramid, 233–234
- Maintenance
 - making test maintenance visible, 213
 - of tests, 248–251
 - visual cues of technical debt, 217
- Maksymchuk, Cory
 - about the contributors section, xli
 - on using examples to prevent defects, 158
- Management 3.0* (Appelo), 16
- Management issues, distributed teams and, 317–318
- Manns, Mary Lynn, 19
- Mapping
 - impact mapping. *See* Impact mapping
 - mind mapping. *See* Mind mapping
 - relationship mapping, 49
 - story mapping. *See* Story mapping
 - tools for structured and focused thinking, 50
- Marick, Brian, 103
 - on automating acceptance tests, 214
 - evolution of agile testing and, 3
 - on example-driven development, 145
 - on learning to code, 57
 - on Quadrants, 101
- Martin, Robert C., (Bob)
 - on building trust, 17
 - on prioritization of high-value work, 401
 - on rules of clean code, 35
- Matrices
 - adding visibility to tests and test results, 390
 - Attribute Component Capability (ACC) matrix, 113–114
 - monitoring risks and assumptions, 92
 - release-level test matrix, 99
- Matts, Chris, 153
 - on Real Options, 18
 - on determining the most valuable options, 123
- McDonald, Mark P., 276, 292
- McKee, Lynn, 25, 55
- McKinney, Drew
 - about the contributors section, xli
 - on user research, 205–206
- McMahon, Chris, 310
- Meetings
 - GoToMeeting collaboration tool, 312
 - for prioritization of work, 364
 - getting the right people, 365
 - standup meetings, 15
- Melnick, Grigori, 214
- Metrics, in goal setting, 277
- Meyer, Geoff, xxix, 287–288, 296–297
 - about the contributors section, xli
 - on agile journey at Dell, 279–282
 - on suitable approach to test automation, 266–267
- Mind mapping
 - adding visibility to tests and test results, 390–391
 - as collaboration tool, 321
 - converting from SBTM to TBTM, 180–181
 - for documentation of deliverables, 94
 - impact mapping developed from, 123
 - mobile apps and, 327
 - for recording results of exploratory testing, 185
 - types of thinking skills, 49
 - for visualization of thinking process, 43
 - for visualizing what you are testing, 98–100
- Minimum viable product (MVP)
 - defined, 418
 - visualizing, 126
- Ministry of Testing, 327
- Mission statement, as source for charters, 169
- Mobile apps/mobile devices
 - automation-assisted exploratory testing (Hagar), 334–335
 - concurrency testing and, 194
 - critical nature of testing in, 328–329
 - integration of hardware and operating systems in, 344–345
 - learning agile testing for (Harrison), 336–337
 - lessons learned in applying agile testing (Hagar), 330–332
 - overview of, 325–326
 - similarities and differences in agile testing in, 326–328
 - test automation, 244
 - test automation strategies (Morgan), 332–334
 - test planning and, 91
 - types of agile approaches used with, 329

Models, data, 347

Models, planning

- agile testing quadrants and, 101–105
- alternative to Quadrants (Hendrickson), 112–113
- challenges to Quadrants, 108–111
- FURPS model and ACC matrix applied to, 113–114
- heuristics applied to, 114
- overview of, 101
- Q1 tests, 105
- Q2 tests, 105–106
- Q3 tests, 106–107
- Q4 tests, 107–108
- test automation and, 115–116

Monitoring

- general technical skills, 59
- managing Big Data, 359
- memory, CPU, logging and, 59
- risks and assumptions, 92
- testing requirements and, 192

Morgan, Jeff (“Cheezy”)

- about the contributors section, xli
- on managing test data, 249–250, 355–356
- on test automation strategies for mobile and embedded systems, 332–334
- on testing browser-based applications, 244–245

Morville, Peter, 328

Moss, Claire

- about the contributors section, xli
- on learning, 80–83

MVP. *See* Minimum viable product (MVP)

N

Naming conventions, test standards, 265

Natural language, use in BDD, 152

North, Dan

- on BDD, 152
- evolution of agile testing and, 3

Notes, recording results of exploratory testing, 185–186

O

OAT (operational acceptance testing)

- cycle time from demo to, 387–389
 - defined, 419
- Observation skills, 48

O’Dell, Chris

- about the contributors section, xliii
- on flipping an inverted test automation pyramid, 229–230

Offshore testing. *See also* Distributed teams

- cultural issues and, 303
- experience working on test team, 315–317
- language usage, 312
- overview of, 301
- pros/cons, 312–315

One-page test plans, 94

Online boards, in visualizing testing for distributed or dispersed teams, 386

Online communities, learning from, 75–77

Online courses, learning from, 74

Online games, for integration of distributed teams, 309

Online mind mapping, visualizing what you are testing, 98–100

Online tracking tools, managing dependencies, 305

Open source projects, as learning opportunity, 76

Operational acceptance testing (OAT)

- cycle time from demo to, 387–389
- defined, 419

Operations

- development and. *See* DevOps
- liability in separating from development, 365

Options Board, for displaying 7 Product

Dimensions, 132

OPV (Other people’s views), thinking skills and, 50

Oracles, in exploratory testing, 166

Organization, of regression tests, 97

Organizational culture

- educating stakeholders, 17–19
- fostering a learning culture, 13–15, 77
- importance of a learning culture, 12–13
- investing time in learning, 8–12
- managing testers, 19–20
- overview of, 7–8
- transparency and feedback loops, 15–17

Organizations

- dealing with organizational controls, 278–283
- enterprise-level. *See* Enterprises (large organizations)

Other people’s views (OPV), thinking skills and, 50

Ottinger, Tim

- Arrange-Act-Assert pattern, 240
- on automating acceptance tests, 214
- mnemonic for nonfunctional requirements—
FURPS, 113

Ownership, offshore test team and, 316

P

Page Object pattern

- creating maintainable UI tests, 245
- creating with PageFactory class, 410–412
- example written with Selenium 2 (WebDriver),
407–410
- implementing, 247–248
- test automation strategies for mobile and
embedded systems, 332–334
- testing through the UI and, 243–246
- understanding, 246–247

Paper prototypes. *See* Prototyping

Parallelization, in managing Big Data, 358–359

Patton, Jeff

- persona use by, 171
- on story mapping, 126–127

Performance tests

- DW/BI and, 357–359
- in expanded test automation pyramid, 233–234
- test environments, 61

Personas

- creating test charters, 171–174
- playing role of “bad customer” in exploratory
testing, 166–167
- in story mapping, 128
- user experience designers creating, 396

“Pillars of Testing” model (Evans), 402–405

Pilot projects, for working out kinks, 280

Planguage (Gilb and Gilb), 107, 277

Planning

- applying to enterprise example, 284
- distributed teams and, 305, 308
- example of lightweight test plan (BMI
calculator), 95–96
- features, 92–94
- feedback/feedback loops, 87
- identifying product needs, 129–131
- iteration planning, 93, 96–97, 130, 293, 310
- iteration planning meetings, 173–174, 186

“Levels of Service” provided by infrastructure
testers, 371

models. *See* Models, planning

points of view in, 87–89

at product level, 286

question asker and, 25

regression testing and, 97–98

release planning, 89–92

story readiness, pre-planning, 96, 293

task level, 96–97

visual (whiteboard) approach to, 81–82

visualizing what you are testing, 98–100

Platform as a service (PaaS), 419

Play, as learning technique, 73–74

Plunkett, Brian, 280

POC (proof of concept), 64

Podcasts, learning from, 75–77

Points of view, in planning, 87–89

Pomodoro Testing (Gärtner), 182–183

“Power of Three” (Dinwiddie), 4

Practice, learning and, 74

Pragmatic Thinking and Learning (Hunt), 49

Prioritization

- of backlog, 173–174
- of defect backlog, 81
- of defect/bug fixes, 289
- of features, 46, 159, 283
- of outcomes, 17
- of stories, 127–128, 293
- of test charters, 396
- as a thinking skill for testing, 41, 49

Privacy, data-related issues, 353

Problem Solving Leadership (PSL) course, 43, 74

Problem solving skills, thinking skills, 43–44

Product Done, taking release to, 287

Product release

- defined, 419
- enterprises and, 296–297
- integrating exploratory testing into agile
testing, 189
- overview of, 88
- planning, 89–92
- regulatory compliance as part of Release
Done, 342
- single and multiple-team product delivery
cycles, 91–92

Products

- roadmap, 90
- test planning at product level, 286

Profiling, general technical skills, 59

Programmers

- buy-in and, 160–161
- on system test team, 284

Programming, 57. *See also* Code/coding

Proof of concept (POC), 64

Prototyping

- example use in, 145
- use in testing, 205–206
- UX designers and testers both using, 140

“Provocation Starters” (Vaage), 71–72, 413–414

Pryce, Nat, 126

Pseudo code, automating tests and, 57

PSL (Problem Solving Leadership) course, 43, 74

Publications, learning from, 75–77

Puppet tools, in DevOps example, 365–367

Putting it all together

- confidence-building practices, 394
- context sensitivity, 399–400
- continual learning, 397–398
- creating shared vision, 402
- example use, 394–395
- exploratory testing, 395–396
- feature testing, 396–397
- keeping it real, 401
- “Pillars of Testing” model (Evans), 402–405
- seven key success factors, 393

Pyramid, test automation

- alternative versions, 224–227
- dangers of delaying test automation, 227–229
- evolution of agile testing at Dell, 282
- example of expanded pyramid, 231–234
- example of flipping an inverted pyramid, 229–231
- original version, 223–224
- overview of, 223
- planning for test automation, 115–116
- showing different dimensions, 231, 234

Q

qTrace tool, for recording, 187–188

Quadrants, agile testing

- alternative to (Hendrickson), 112–113

A/B tests in Q2. *See* A/B tests

challenges to, 108–111

DevOps working across all, 363–364

evolution of agile testing at Dell, 282

order of applying, 103–105, 191, 239

overview of, 101–103

Q1 tests, 105, 145

Q2 tests, 105–106, 145

Q3 tests, 106–107

Q4 tests, 107–108, 268

technical debt due to skipping Q3 and

Q4 tests, 211

testing quality attributes, 65–67

UAT in Q3, 201

Quality

balancing many aspects of, 135

DevOps and, 363

guilds and, 14

Quality attributes

in 7 Product Dimensions, 130

in acceptance tests, 150

representation in operations and development dimensions, 191

testing, 65–67

Query languages, skill requirements, 59

Questioning (tools)

tools for visualization

of thinking process, 44

“Why?,” “Who?,” “How?,” and “What?,” 123–124

Queueing theory, capacity utilization and, 10–12

R

Rainsberger, J. B., 147

Rall, Aldo, xxix

about the contributors section, xli–xlii

on helping others, 79

Rasmussen, Jonathan, xxi

Real Options (Mattis and Maassen), 18

Reality checks. *See* Keeping it real

Recording

refactoring ideas, on whiteboards, 230

results of exploratory testing, 185–188

Refactoring

defined, 419

scaling automation for large organizations, 268

whiteboard for recording ideas, 230

- Reflect-and-adapt (Hendrickson), xviii
- Regression tests
 - challenges of, 200–201
 - continuous integration with automated tests, 289
 - defined, 419
 - DevOps role building/deploying, 376
 - DW/BI and, 349
 - in expanded test automation pyramid, 234
 - failures, 62
 - key success factors, 393
 - managing with SBTM charters, 177–178
 - manual, 213, 223
 - planning for, 97–98
 - scaling for large organizations, 267–268
 - team approach to biggest problem, 217–218
 - technical debt in, 216–217
 - testing through the UI and, 243, 245
 - too many, 161
- Regulatory environments
 - agile use in regulated environments, 343
 - collaboration in meeting regulatory needs, 344–346
 - compliance and, 340–341
 - excessive documentation and, 341–342
 - including auditors in the solution, 342
 - “lack of documentation” myth, 339–340
 - overview of, 339
- Relationship mapping, types of thinking skills, 49
- Relationship skills, 74
- Release candidates
 - creating continuously, 365
 - defined, 419
 - product release level and, 189
- Release Done, regulatory compliance as part of, 342
- Release planning sessions, 92
- Reliability, in expanded test automation pyramid, 233–234
- Requirements gathering, 159
- Resources, for learning
 - conferences, courses, meet-ups, and collaborating, 72–75
 - publications, podcasts, and online communities, 75–77
- Retrospectives
 - in addressing technical debt, 217
 - collaboration tools, 322
 - power of, 13
 - self-managed teams and, 24
- Return on investment (ROI)
 - defined, 419
 - key components of business value of software, 122
 - meeting automation challenges and, 258
 - test automation pyramid and, 115
 - test planning and, 91
 - testing through the UI and, 243
- Ries, Eric, 134
- Rising, Linda
 - Fearless Change (Manns and Rising), 19
 - on power of retrospectives, 13
- Risk-based testing, 51
- Risks, testing, 125
- R-mode (nonlinear, fast, “rich”), thinking skills and, 49
- Roadmaps, questions in creation of, 123–124
- Robson, Sharon, xxix
 - about the contributors section, xlii
 - adding dimensions to test automation pyramid, 231–232
 - on application of thinking skills to testing, 49–51
 - on effective collaboration, 52–53
- Rogalsky, Steve, xxix
 - about the contributors section, xlii
 - on learning, 79
 - on podcast as learning resource, 75–76
 - on story mapping and testing, 127–129
 - on use of kanban in testing, 382–385
- ROI. *See* Return on investment (ROI)
- Roles/competencies
 - advantages of some blurred roles on teams, 142
 - competencies vs. roles, 24–28
 - generalizing specialists, 33–36
 - hiring the right people, 36–37
 - onboarding process for testers, 37–38
 - overview of, 23
 - square-shaped team and, 30–32
 - T-shaped skill set and, 28–30
 - importance of titles (Walen), 25

Root-cause analysis, tools for visualization of thinking process, 44

Rothman, Johanna, xix

- on feedback loops and transparency, 16
- on hiring right people, 37

RSpec approach, to test-driven development, 259

Rubin, Ken, xxix

Ruby Faker library, 250

Ruhland, Bernice Niel, xxix, 77

- about the contributors section, xlii
- on adapting to change, 15
- on group exploration process, 183
- on maintaining list of testing types for her team, 193
- on one-page test plans, 94
- on recording results of exploratory testing, 187
- on SBTM use to train testers, 176
- on self-managed teams, 23–24
- on testers and business analysts, 138

S

SaaS (software as a service)

- defined, 420
- product development and, 202

Sandboxes, test environments, 60

Satir Global Network, 74

SBE. *See* Specification by example (SBE)

SBMT. *See* Session-based test management (SBTM)

Scale, in agile testing

- automation and, 358–359
- considerations regarding, 275
- Dell example, 287–288
- for enterprises, 276–277
- speed and, 357–358

Scenarios

- creating test charters, 174
- use of examples and, 159

Schoots, Huib

- about the contributors section, xliii
- on documentation, 94
- experience working on distributed teams, 306–307

Scope

- adding infrastructure to testing scope, 365–367
- deriving using SBE, 153

Scott, Alister

- alternative test automation pyramid, 227
- on testing through the UI, 243

Scripted tests, 166

Scrum/scrum teams

- Agile evolution at Dell and, 282
- in automated build verification, 370
- Extreme Manufacturing and, 331
- first agile experiences compared with, 382
- scaling agile testing, 287–288
- sprint cycles, 266–267
- time-boxing, 281

Security

- data-related issues, 353
- security worried persona, 172
- applying Q4 tests, 103
- quality attributes and, 65–66
- skill requirements for DW/BI testers, 348

Selenium 2 (WebDriver)

- Page Object example, 407–410
- PageFactory class, 410–412

Selenium test library, 244, 254–255, 265

Self-managed teams, Ruhland on, 23–24

Semantic Studios, 329

Session sheets, recording results of exploratory testing, 185

Session-based test management (SBTM)

- converting to TBTM, 179–181
- managing regression tests with SBTM charters, 177–178
- managing test charters, 176, 178
- charters and session-based testing, 169–170
- recording results of exploratory testing, 186

Set-based development, 245, 259, 420

Shannon, Paul

- about the contributors section, xliii
- on flipping an inverted test automation pyramid, 229–230

“Sharing the pain,” 213

Shore, James, 214

Simplicity principle

- applying to DW/BI, 351
- applying to planning, 88
- automated tests and, 256
- mind mapping for detail capture, 94
- reviewing workflow for simplicity, 385

- Sinclair, Jennifer, xxix, xliii
- Sinclair, Toby
- about the contributors section, xliii
 - on A/B tests, 203–204
- Single, responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion (SOLID), 57
- Sjödahl, Lars, 304
- Skills
- business analysis skills, 137–139
 - DevOps activities and, 363
 - documentation skills, 141–142
 - investing time in learning, 8
 - thinking skills. *See* Thinking skills
 - square-shaped team example, 30–32
 - team building and, 24, 27–28
 - technical skills, *See* Technical awareness (technical skills)
 - T-shaped skill set, 28–30
 - UX design skills, 140–141
- Slices
- feature testing and, 396
 - testing, 127
- Small chunks principle
- applying to DW/BI, 351
 - build-measure-learn (BML) and, 134
 - charters as, 170
 - distributed teams and, 313
 - identifying chunks for testing, 127
 - for learning and adapting, 87
 - for planning, 87
 - stories and, 88, 103
 - for training, 38
- SMART (specific, measurable, achievable, relevant, and time-boxed)
- continuous improvement and, 386
 - types of thinking skills, 49
- Smoke tests, 263
- Socratic questioning, types of thinking skills, 49
- Soft skills. *See* Thinking skills
- Software as a service (SaaS)
- defined, 420
 - product development and, 202
- Software delivery cycle, feedback loops in, 15
- Software Test Attacks to Break Mobile and Embedded Devices* (Hagar), 330
- SOLID: Single, responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion (SOLID), 57
- Solving problems, 43–44
- SonarQube, checking business code with, 366
- Source code control system, 62–65, 420
- tester expertise, 60
- version control, 61
- SOX compliance, regulatory environments, 340
- Specialists, generalizing specialists, 33–36
- Specification by Example* (Adzic), 56
- Specification by example (SBE)
- agile approaches used with mobile and embedded systems, 329
 - confidence-building practices, 394
 - defined, 420
 - functional test tools, 237
 - guiding development with examples, 55–56, 148, 153–154
 - use in reducing defect debt, 214
 - in whole-team approach to meeting new challenges, 259
- Specification workshops, facilitating, 42
- Spikes
- deferring test planning until after spike solutions, 92
 - planning and, 90
 - testing potential automation tool with, 261
- Split testing. *See* A/B tests
- Spotify, 16, 297
- Spott, Dave, 280
- Spreadsheets
- example use in, 145, 157
 - recording results of exploratory testing, 186
 - struggling with failing CI builds, 263
- SQL (Structured Query Language), 59
- SQL injection, security tests, 65–66
- Square-shaped team, 30–32
- Staging environments, for testing, 60
- Stakeholders
- accountability to, 343
 - auditors as, 342
 - buy-in, 160–161
 - educating, 17–19
 - getting examples from, 147–148
 - giving/receiving feedback, 46

Stakeholders (*continued*)

- specification workshops and, 42
- tools for engagement with, 123–127

Standards, naming conventions, 265

Standup meetings, for face-to-face communication, 15

State transition diagrams, 67

Steel threads, 160, 313

Stories

- blocked, 292
- breaking features up into, 92, 94
- in coaching, 48
- for code refactoring, 213
- creating test charters from, 175
- determining purpose of, 121
- feature-testing, 94, 96
- getting examples for, 155
- guiding development with examples, 55
- iteration testing and, 28
- in levels of precision, 89
- levels of precision in planning, 96
- overview of, 88–89
- planning Q2 tests and, 105
- planning Q3 tests and, 106–107
- prioritizing, 127–128, 293
- regression testing and, 97–98
- team approach to biggest problem, 218

Story boards

- communication tools, 319
- effective visualization of testing process, 384–386
- getting to know distributed team members, 311

Story Done, vs. Feature Done, 286

Story mapping

- getting examples and, 155
- impact mapping developed from, 123
- overview of, 126–127
- testing and, 127–129

Stress tests

- meeting regulatory needs and, 345
- mobile devices and, 345

Structured Query Language (SQL), 59

Subversion, 64

Success

- importance of celebrating, 17
- key factors in, 393

Sweets, Tony

- about the contributors section, xliii–xliv
- on use of Page Object pattern, 246–248

System integration team, approaches to offshore testing, 313

System test team

- coordinating dependencies between teams, 285–286
- creating, 284–285
- managing, 287

System under test (SUT)

- changes and updates and, 240
- how much automation is enough and, 263

T

Tabaka, Jean, 71

Tables, recording results of exploratory testing, 187

Talks, Mike, xxix

- about the contributors section, xliv
- on constant learning, 78–79
- on downside of kanban boards, 384
- example of need for business analysis in testing, 138–139
- on learning domain knowledge, 46–47
- on onboarding process for new testers, 38

Tasks

- addressing both product and regulatory needs, 342
- creating testing tasks, 96–97
- exploratory testing with, 189

Taxonomy, Quadrants as, 104

TBMT. *See* Thread-based test management (TBMT)TDD. *See* Test-driven development (TDD)

Teams

- adopting agile values, 7
- automation solutions for teams in transition, 253–254, 258
- bonding with, 345
- competencies included on, 27
- coordinating multiple, 283–284
- creating system test team, 284–289
- dependencies between, 92
- dispersed. *See* Dispersed teams
- distributed. *See* Distributed teams

- finding/hiring the right people, 36–37
- fostering a learning culture, 13–15
- generalizing specialists and, 36
- including business analysts on, 129
- multidisciplinary, 256
- product delivery cycles and, 91
- self-managed (Ruhland), 23–24
- square-shaped team, 30–32
- technical skills and, 56
- understanding team issues (Gregory), 18
- whole-team approach. *See* Whole team
- Technical awareness (technical skills)
 - automation and coding skills, 56–58
 - automation through the API, 240–243
 - automation through the UI, 243
 - continuous integration and source code control systems, 62–65
 - development environments and, 59–60
 - DW/BI testing, 348
 - general technical skills, 59
 - guiding development with examples, 55–56
 - learning resources, 74
 - overview of, 55
 - quality attribute testing, 65–67
 - test design techniques, 67
 - test environments, 60–62
- Technical debt
 - defined, 420
 - making it visible, 212–213, 216–217
 - overview of, 211–212
 - quantifying cost of, 17
 - reducing defect debt, 213–216
 - reducing through collaboration, 218–220
 - team approach to, 217–218, 220
 - test automation volcano and, 229
 - unrealistic deadlines creating, 7–8
- Technical skills. *See* Technical awareness (technical skills)
- Technical writers, 141–142
- Telecommuting, creating policy for, 310
- Test automation
 - acceptance tests, 214
 - aligning target setting for test automation across enterprise, 277–278
 - automation-assisted exploratory testing (Hagar), 334–335
 - collaborative solutions in tool selection, 260–261, 264
 - continuous integration with automated regression testing, 289
 - data management, 249–250
 - design patterns and principles, 240
 - development environments and, 59–60
 - examples turned into automated tests, 147
 - exploratory testing used in conjunction with, 167
 - how much automation is enough, 262–263
 - implementing in large organizations, 254–258
 - maintaining tests, 248–251
 - for mobile and embedded systems, 332–334
 - overview of, 209–210
 - Page Object pattern and, 246–248
 - Planning, 115–116
 - pyramid approaches. *See* Pyramid, test automation
 - Q4 tests (technology-facing tests), 268
 - regression testing, 97
 - rules and reasons for, 241
 - scaling for large organizations, 264–265, 267–268
 - scaling for Big Data, 357–359
 - selecting test automation solutions, 253
 - solutions for teams in transition, 253–254, 258
 - starting off right, 239–241
 - technical debt and. *See* Technical debt
 - technical skills for, 56–58
 - test environments and, 61
 - testing through the API (service level), 241–243
 - testing through the UI, 243–246
 - third-party partnerships and, 293
 - tools and processes in, 237–238
 - transition challenges and, 266–267
 - whole team approach to, 238–239, 258–260
- Test charters. *See* Charters
- Test coverage
 - in enterprises, 291
 - medical device example, 344
- Test design. *See also* Design patterns and principles techniques, 67
 - use of diagrams for, 67
 - UX design skills, 140–141
 - UX designers creating personas, 396

Test environments. *See also* Environments

- build pipelines and, 367
- building and maintaining, 60–62
- DevOps in maintenance of, 362
- making visible, 291
- overview of, 60–62

Test failure

- debugging, 250–251
- managing, 289–290
- timing issues, 237

“Test Heuristics Cheat Sheet” (Hendrickson), 114

Test matrix. *See* Matrices

Test-driven design. *See* Test-driven development (TDD)

Test-Driven Development (Beck), 9

Test-driven development (TDD)

- BDD as response to, 153
- collaborating with distributed teams, 312
- core development practices, 394
- defined, 421
- DSL examples and, 158
- example-driven development compared with, 148
- integrating coding and testing, 105
- in mobile and embedded systems, 326, 329
- programmer buy-in and, 160
- reducing defect debt, 214
- scaling, 276–277
- at the task level, 96, 98

Testers. *See also* Roles/competencies

- adding DevOps value, 371–372
- agile principles for, 28–29, 386
- business analysis skills, 137–140
- as community of practice (CoP), 80
- database skills, 348
- determining job responsibilities, 137
- documentation skills, 141–142
- generalizing specialists and, 36
- giving/receiving feedback and, 46
- “Levels of Service” provided by infrastructure testers, 371–372
- managing, 19–20
- need for in development (Khoo), 26–27
- onboarding process for, 37–38
- quality guilds (Evangelisti), 14
- SBTM use in training, 176

square-shaped team example, 30–32

on system test team, 284

taking initiative, 142–143

technical skills and collaboration, 56

Trio (developer, tester, BA) in exploratory testing, 184

UX design skills, 140–141

Testing

acceptance tests. *See* Acceptance tests

A/B tests. *See* A/B tests

BI tests. *See* Business intelligence (BI) tests

business analysis overlapping with, 137

component tests, 224–225, 233–234

context-driven, 399

creating tasks for, 96–97

creating tests before coding, 96

early, 134

end-to-end tests. *See* End-to-end testing

evolution of agile testing, 3–6

example of lightweight test plan (BMI calculator), 95–96

exploratory. *See* Exploratory testing

features. *See* Feature testing

functional tests. *See* Functional tests

integrating coding with, 105

investigative. *See* Investigative testing

learning, 72–76

load testing, 61, 103

offshore. *See* Offshore testing

performance tests. *See* Performance tests

Pomodoro testing (Gärtner), 182–183

prototyping in, 205–206

Quadrants. *See* Quadrants, agile testing

quality attributes, 65–67

regression tests. *See* Regression tests

requirements and purpose of (Walen), 139–140

risk-based, 51

risks, 125

scaling tests, 276–277, 287–288

scripted tests, 166

session-based, 169–170

slices, 127

smoke tests, 263

as social science (Walen), 25

staging environments for, 60

starting process right (Gärtner), 239

- story mapping and, 127–129
- stress tests, 345
- testing-as-a-service, 314
- thinking skills applied to (Robson), 49–51
- through the API (service level), 241–243
- through the UI, 243–246
- types of, 192–194
- unit tests. *See* Unit tests
- user acceptance testing (UAT), 149–150
- visualizing what you are testing, 98–100
- workflow tests. *See* Workflow tests
- Testing, in enterprise
 - aligning target setting for test automation, 277–278
 - implementing test automation in, 254–258
 - scaling testing, 276–277
 - system test team and environment and, 284–289
 - test coverage, 291
- Testing in production (TiP) (Eliot), 200–201
- Testing-as-a-service, 314
- Thinking skills
 - coaching and listening, 48–49
 - collaboration skills, 52–53
 - diagrams for structured and focused thinking (Robson), 50
 - domain knowledge and, 46–47
 - exploratory testing and, 165
 - facilitation, 42–43
 - giving/receiving feedback, 45–46
 - learning resources, 75
 - organizational skills, 51
 - overview of, 41–42
 - problem solving, 43–44
 - thinking differently, 49–51
- Thinking, Fast and Slow* (Kahneman), 50
- Thinking for Action* (de Bono), 50
- “The Thinking Tester” (Hendrickson), 112
- Third-party
 - compatibility issues, 178
 - test libraries, 244
- Third-party vendors
 - customer involvement in enterprises, 294–296
 - managing dependencies, 292–294
 - time frames and expectations of, 200
 - “why” of feature development and, 122
- Thread-based test management (TBMT)
 - converting from SBMT to, 179–181
 - fractal representation of, 182
 - overview of, 178, 181–183
- “Three Amigos” (Dinwiddie), 4
- Thucydides, for automating acceptance tests, 214
- Time constraints, organizational skills and, 51
- Time management, making time for learning, 77–79
- Time zones
 - communication and collaboration and, 309–310
 - coping strategies for distributed teams, 309
 - issues facing distributed teams, 304–305
- Time-boxing
 - in collaboration process, 53
 - Scrum guidelines and, 281
 - story level and, 96
 - test planning and, 88
- Titles, 25
- Tools. *See also* by individual types
 - Agile Alliance Functional Test Tools committee (AA-FTT), 4, 153
 - analytic tools, 327
 - “bake-offs” for testing, 261–262
 - collaboration tools, 312, 322
 - collaborative approach to selecting, 264
 - consistency in selection of, 265, 289
 - coordination of tooling, 289–290
 - for DevOps, 366, 373
 - for exploring early, 134
 - facilitating online communication, 306–307
 - implementing agile at Dell and, 281
 - recording tools, 187–188
 - in test automation, 237–238
 - tester expertise with IDE tools, 60
 - thinking tools, 49–50
 - videoconferencing tools, 315–317
 - for visualization of thinking process, 43–44
- Tools, for customer engagement
 - 7 Product Dimensions (Gottesdiener and Gorman), 129–133
 - impact mapping, 123–126
 - overview of, 123
 - story mapping, 126–129
- Tools, for distributed teams
 - collaboration tools, 319–321
 - communication tools, 319

Tours, creating test charters, 174–175
 Transition, to agile development
 automation solutions for teams in transition, 253–254, 258
 challenges, 266–267
 Transparency, in organizational culture, 15–17. *See also* Visibility
 “Trinity Testing” (Harty), 184
 Trust
 building, 17
 self-managed teams and, 24
 T-shaped skills
 collaboration as means of developing, 344
 developing, 398
 generalizing specialists and, 33
 roles and competencies and, 28–30
 Tung, Portia, 73–74

U

UAT. *See* User acceptance testing (UAT)
 UIs. *See* User interface (UI)
 Uncertainty, building trust and, 18
 Unit tests
 consistency in tool selection and, 265
 defined, 421
 DW/BI and, 349
 JUnit tests, 265
 planning early and automating, 105
 in TDD, 98
 test automation pyramids and, 115, 224–225, 233–234
 UNIX shell commands, skill requirements, 59
 Usability feedback
 in expanded test automation pyramid, 233–234
 from prototyping, 205–206
 User acceptance testing (UAT)
 acceptance tests contrasted with, 149–150
 in enterprises (Bligh), 294–295
 in expanded test automation pyramid, 233–234
 learning to test BI and, 351–352
 offshore testing and, 312–314
 overview of, 201–202
 product release level and, 90
 Q3 tests and, 107
 value of conversations and, 151–152

User experience (UX)
 A/B tests applied to, 203
 considering impact of stakeholders on business goals, 124
 design skill requirements, 140–141
 mobile devices and, 328–329
 testing, 205–207
 UX designers creating personas, 396
 User interface (UI)
 achieving consensus for automation solutions, 260–261
 collaboration in meeting regulatory needs, 344
 consistency in tool selection and, 265
 example of benefit of including UX design in tests, 141
 mobile devices and, 328
 technical skills and, 58
 test automation and, 225–226, 287
 in test automation pyramid, 223–224
 testing mobile projects and, 344
 testing through the UI, 243–246
 User stories. *See* Stories
User Story Mapping (Patton), 127
 Users
 in 7 Product Dimensions, 129
 training end users, 295
 Utilization, capacity utilization, 10–12
 UX. *See* User experience (UX)

V

Vaage, Carol, 71–72, 413–414
 Vagrant tool, used in DevOps example, 366
 Vendors. *See* Third-party vendors
 Version control
 in enterprises, 290
 Git and, 365
 source code control systems, 61–65, 420
 Videoconferencing
 collaboration tools, 319–320
 working on offshore test team and, 315–317
 Virtual machines, for running test suites, 262
 Visibility
 communicating importance of testing, 381–382
 diagrams for visualization of thinking process, 44
 for continuous improvement, 386–390

giving/receiving feedback and, 46
 kanban and, 382–385
 online boards for distributed or dispersed teams, 386
 overview of, 381
 reducing need for control, 16
 story boards and, 384–386
 of technical debt, 212–213, 216–217
 of test environments, 291
 of tests and test results, 381, 390–392
 working on offshore test team and, 316–317
 Vision, creating shared, 402
 Visual aids, 98–100
 Visual learning, 69, 81
 Visualization. *See* Visibility
 Volcano, test automation, 228
 Vuolli, Eveliina
 about the contributors section, xlv
 on introducing automation in enterprise system, 277–278

W

Walen, Pete
 about the contributors section, xlv
 on importance of titles, 25
 on requirements and purpose of testing, 139–140
 “Walking skeleton” (Freeman and Pryce), 126
 Walshe, Mary
 about the contributors section, xlv–xlv
 on kanban for continuous improvement, 386–389
 Web browsers. *See* Browsers
 WebDriver tool, 261
 WebEx, working on offshore test team and, 315–317
 Weekend Testing, 74
 “What?” questions
 in creation of roadmaps, 123–124
 iPhone example, 140
 “what” vs. “why” in feature development, 122
 Whelan, Declan, 153
 Whittaker, James, 109
 “Who?” questions
 in creation of impact maps, 123–124
 iPhone example impact map, 140
 Whole team. *See also* DevOps
 achieving consensus for automation solutions, 260–262
 approach to agile testing, 362
 approach to test automation, 238–239
 key success factors, 393
 meeting automation challenges, 228, 258–260
 required for DW/BI testing, 348–349
 understanding operating environment, 376
 “Why?” questions
 in building the right thing. *See* Development, building the right thing
 iPhone example impact map, 140
 questions in creation of impact maps, 123–124
 Wiedemann, Christin
 about the contributors section, xlv
 on converting from SBTM to TBTM, 179–181
 Wikis
 adding visibility to tests and test results, 392
 automation solutions in large organizations, 257
 collaboration tools, 321
 communication tools, 319
 integration of distributed teams, 309
 recording results of exploratory testing, 185
 team wikis, 36
 Winterboer, Lynn
 about the contributors section, xlv
 on solving the bad test data problem, 353–354
 WIP (work-in-progress)
 bandwidth for testing and, 401
 kanban and, 382
 Workflow tests
 automating provisioning of configuration base states, 374–376
 in expanded test automation pyramid, 233–234
 levels for testing through the UI, 243
 in test automation pyramid, 115–116
 Work-in-progress *See* WIP
 Workshops
 getting participation in specification workshops, 155
 as learning resource, 74
 Wortel, Cirilo, 265
 about the contributors section, xlv

Wortel, Cirilo (*continued*)

on automation solutions in large organizations,
254–258

Wynne, Mat, 56

X

XP. *See* Extreme Programming (XP)

Z

“Zen, the Beginner’s Mind” workshop (Hussman
and Tabaka), 71

Zheglov, Alexei

about the contributors section, xlv

on capacity utilization, 10

This page intentionally left blank

PEARSON

InformIT is a brand of Pearson and the online presence for the world's leading technology publishers. It's your source for reliable and qualified content and knowledge, providing access to the top brands, authors, and contributors from the tech community.

Addison-Wesley

Cisco Press

EXAM **CRAM**

IBM
Press

QUE

PRENTICE
HALL

SAMS

Safari
Books Online

LearnIT at InformIT

Looking for a book, eBook, or training video on a new technology? Seeking timely and relevant information and tutorials? Looking for expert opinions, advice, and tips? **InformIT has the solution.**

- Learn about new releases and special promotions by subscribing to a wide variety of newsletters. Visit **informit.com/newsletters**.
- Access FREE podcasts from experts at **informit.com/podcasts**.
- Read the latest author articles and sample chapters at **informit.com/articles**.
- Access thousands of books and videos in the Safari Books Online digital library at **safari.informit.com**.
- Get tips from expert blogs at **informit.com/blogs**.

Visit **informit.com/learn** to discover all the ways you can access the hottest technology content.

Are You Part of the IT Crowd?

Connect with Pearson authors and editors via RSS feeds, Facebook, Twitter, YouTube, and more! Visit **informit.com/socialconnect**.



REGISTER



THIS PRODUCT

informit.com/register

Register the Addison-Wesley, Exam Cram, Prentice Hall, Que, and Sams products you own to unlock great benefits.

To begin the registration process, simply go to **informit.com/register** to sign in or create an account.

You will then be prompted to enter the 10- or 13-digit ISBN that appears on the back cover of your product.

Registering your products can unlock the following benefits:

- Access to supplemental content, including bonus chapters, source code, or project files.
- A coupon to be used on your next purchase.

Registration benefits vary by product. Benefits will be listed on your Account page under Registered Products.

About InformIT — THE TRUSTED TECHNOLOGY LEARNING SOURCE

INFORMIT IS HOME TO THE LEADING TECHNOLOGY PUBLISHING IMPRINTS Addison-Wesley Professional, Cisco Press, Exam Cram, IBM Press, Prentice Hall Professional, Que, and Sams. Here you will gain access to quality and trusted content and resources from the authors, creators, innovators, and leaders of technology. Whether you're looking for a book on a new technology, a helpful article, timely newsletters, or access to the Safari Books Online digital library, InformIT has a solution for you.

informIT.com

THE TRUSTED TECHNOLOGY LEARNING SOURCE

Addison-Wesley | Cisco Press | Exam Cram
IBM Press | Que | Prentice Hall | Sams

SAFARI BOOKS ONLINE